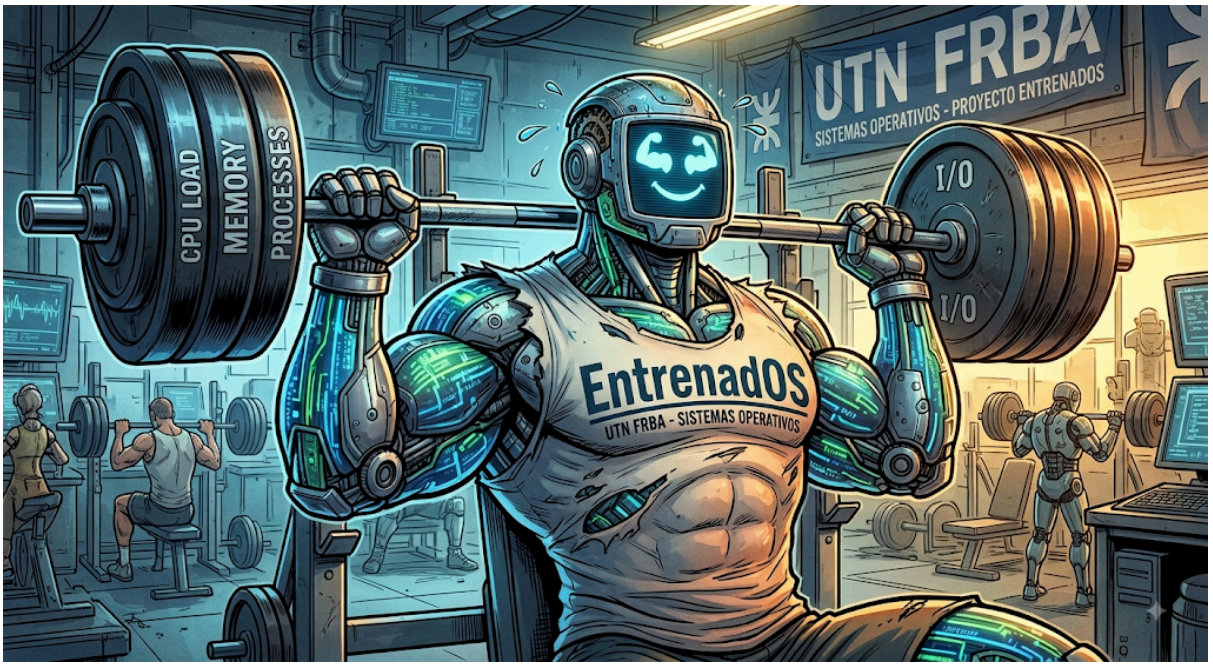
EntrenadOS

Diseño e Implementación de un Sistema Distribuido



¿Cuánto es tu PR en Commits funcionales?

Cátedra de Sistemas Operativos

Trabajo práctico Cuatrimestral

- 2C2026 -
Versión 1.0

Índice

Índice **2**
Historial de Cambios **5**
Objetivos del Trabajo Práctico **6**
Características 6
Evaluación del Trabajo Práctico 6
Deployment y Testing del Trabajo Práctico 7
Definición del Trabajo Práctico **8**
¿Qué es el trabajo práctico y cómo empezamos? 8
Arquitectura del sistema 9
Aclaración importante 10
Módulo: Planificador **10**
Lineamiento e Implementación 10
Planificación de Jobs 10
Contexto de Ejecución 11
Planificación de Largo Plazo 11
Planificación de Corto Plazo 11
Page Fault 11
Atención de Syscalls 11
Jobs 12
Memoria 12
Servicios 12
Checkpoints 12
Servicios 13
Loader 13
Labeler 13
Logger 13
Estadísticas 13
Logs mínimos y obligatorios 14
Archivo de Configuración 14
Ejemplo de Archivo de Configuración 15
Módulo: Core **16**
Lineamiento e Implementación 16
Registros del Core 16
Ciclo de Instrucción 17
Fetch 17
Decode 17
MMU 17
Listado de instrucciones a interpretar 18
Execute 18
Instrucciones básicas 19
Instrucciones de entrenamiento 19
Instrucciones de tipo syscall 19
Check Interrupt 20
Ejemplo de un Job 20
Logs mínimos y obligatorios 21
Archivo de Configuración 21
Ejemplo de Archivo de Configuración 22

Módulo: Placa **23**
Lineamiento e Implementación 23
Memoria de Instrucciones 23
Memoria de Usuario 23
Esquema de memoria y Estructuras 23
El Offload 24
Reemplazo de páginas 24
Operaciones 24
Creación de Job 24
Asignación de espacio 24
Liberación de espacio 24
Obtención de marco 25
Carga de página 25
Lectura de datos 25
Escritura de datos 25
Deslockeo de páginas 25
Finalización de Job 25
Comandos locales 25
Logs mínimos y obligatorios 26
Archivo de Configuración 26
Ejemplo de Archivo de Configuración 27
Módulo: Storage **28**
Lineamiento e Implementación 28
Implementación 28
Estructura interna del sistema de archivos 28
Superbloque 28
File Allocation Table 29
Directorio 29
Operaciones 29
Guardar un Checkpoint 29
Cargar un Checkpoint 30
Eliminar un Checkpoint 30
Journaling 30
El journal 30
Procedimiento de uso 30
Primer paso: escritura en el journal 30
Segundo paso: escritura en filesystem 31
Recuperación ante fallas 31
Ejemplo de un Asiento en el Journal 31
Comandos locales 32
Logs mínimos y obligatorios 32
Archivo de Configuración 32
Ejemplo de Archivo de Configuración 33
Descripción de las entregas **34**
Check de Control Obligatorio 1: Conexiones y Serialización 34
Check de Control Obligatorio 2: Planificación y Ejecución Básica 34
Check de Control Obligatorio 3: Memoria, Paginación y Servicios, Storage 34
Check de Control Obligatorio 4: Storage, Filesystem y Persistencia 35
Entregas Finales 35

Historial de Cambios

v1.0 (25/08/2026) Publicación del enunciado.

Objetivos del Trabajo Práctico

Mediante la realización de este trabajo se espera que el alumno:

- Adquiera conceptos prácticos del uso de las distintas herramientas de programación e interfaces (APIs) que brindan los sistemas operativos.
- Entienda aspectos del diseño de un sistema operativo.
- Afirme diversos conceptos teóricos de la materia mediante la implementación práctica de algunos de ellos.
- Se familiarice con técnicas de diseño y programación de sistemas distribuidos, así como también con el uso de archivos de configuración y archivos de log.

Debido al fin académico del trabajo práctico, los conceptos que se verán reflejados son, en general, versiones simplificadas o alteradas de los componentes reales de hardware y de sistemas operativos vistos en las clases, a fin de resaltar aspectos de diseño o simplificar su implementación.

Invitamos a los alumnos a leer las notas y comentarios al respecto que haya en el enunciado, reflexionar y discutir con sus compañeros, ayudantes y docentes al respecto.

Características

- Modalidad: grupal (5 integrantes ± 0) y obligatorio
- Fecha de comienzo: 25/08
- Fecha de primera entrega: 28/11
- Fecha de segunda entrega: 12/12
- Fecha de tercera entrega: 19/12
- Lugar de corrección: Laboratorio de Sistemas - Medrano.

Evaluación del Trabajo Práctico

El trabajo práctico consta de una evaluación en 2 etapas.

La primera etapa consistirá en las pruebas de los programas desarrollados en el laboratorio. Previo a la evaluación final se subirán una serie de pruebas para que los alumnos puedan validar el correcto funcionamiento de su trabajo práctico. Queda aclarado que para que un trabajo práctico sea considerado evaluable, el mismo debe proporcionar logs de su funcionamiento de la forma más clara posible, para ello se les proveerá en cada módulo un listado de logs mínimos y obligatorios.

La segunda etapa se dará en caso de aprobada la primera y constará de un coloquio individual, con el objetivo de validar y afianzar la aplicación de los conocimientos adquiridos durante el desarrollo del trabajo práctico y definir la nota de cada uno de los integrantes del grupo, por lo que se recomienda que la carga de trabajo se distribuya de la manera más equitativa posible.

Cabe aclarar que el trabajo equitativo no asegura la aprobación de la totalidad de los integrantes, sino que cada uno tendrá que defender y explicar tanto teórica como prácticamente lo desarrollado y aprendido a lo largo de la cursada.

La defensa del trabajo práctico (o coloquio) consta de la relación de lo visto durante la teoría con lo implementado. De esta manera, una implementación que contradiga lo visto en clase o lo escrito en el documento *es motivo de desaprobación del trabajo práctico*. Dicha etapa *no es recuperable*.

Deployment y Testing del Trabajo Práctico

El trabajo implicará el desarrollo de un sistema distribuido, de manera tal que los diversos módulos (Procesos) deberán poder ser ejecutados en diferentes computadoras. La cantidad de computadoras involucradas y la distribución de los diversos procesos en estas será definida en cada uno de los tests de la evaluación y es posible cambiar la misma en el momento de la evaluación. Es responsabilidad del grupo automatizar el despliegue de los diversos procesos con sus correspondientes archivos de configuración para cada uno de los diversos tests a evaluar.

Todo esto estará detallado en el documento de pruebas que se publicará cercano a la fecha de Entrega Final. Archivos y programas de ejemplo se pueden encontrar en el repositorio de la cátedra.

Finalmente, es mandatoria la lectura y entendimiento de las Normas del Trabajo Práctico donde se especifican todos los lineamientos de cómo se desarrollará la materia durante el cuatrimestre.

Definición del Trabajo Práctico

Esta sección se compone de una introducción y definición de carácter global sobre el trabajo práctico. Posteriormente se explicarán por separado cada uno de los distintos módulos que lo componen, pudiéndose encontrar los siguientes títulos:

- **Lineamiento e Implementación**: Contendrá la definición funcional y aspectos de implementación técnica obligatorios del módulo en cuestión. La no inclusión de alguno de los puntos especificados en este título puede conllevar a la desaprobación del trabajo práctico.

- **Archivos de Configuración**: Se describirán los parámetros mínimos requeridos para ajustar el comportamiento de cada módulo ante cada prueba, sin recompilar. Durante la evaluación, deberá alcanzar con detener la ejecución, modificar el archivo de configuración y volver a ejecutar el módulo. En caso de que el grupo requiera de algún parámetro extra, podrá agregarlo.

Cada módulo contará con un listado de **logs mínimos y obligatorios** los cuales deberán realizarse utilizando la biblioteca de so-commons-library provista por la cátedra y los mismos deberán estar como LOG_LEVEL_INFO, pudiendo ser extendidos por necesidad del grupo utilizando LOG_LEVEL_DEBUG. En la descripción de los **logs mínimos y obligatorios**, se utiliza la notación de <> para encerrar los valores que irán variando de acuerdo a la ejecución, pero los caracteres menor y mayor (< >) no deberán estar presentes en el log final.

En caso de no cumplir con los logs mínimos y/o no guardarlos en archivo, *se considerará que el TP no es apto para ser evaluado* y por consecuencia el mismo estará *desaprobado*.

Cabe destacar que en ciertos puntos de este enunciado se explicarán exactamente cómo deben ser las funcionalidades a desarrollar, mientras que en otros no se definirá específicamente, quedando su implementación a decisión y definición del equipo. Se recomienda en estos casos siempre consultar en el foro de github, dado que las justificaciones deberán ser expuestas en el coloquio.

¿Qué es el trabajo práctico y cómo empezamos?

El objetivo del trabajo práctico consiste en desarrollar una solución que permita la simulación de un sistema operativo, a través de un sistema distribuido, donde los grupos tendrán que planificar Procesos, resolver peticiones al sistema, y administrar de manera adecuada tanto una Memoria como un Filesystem (o Sistema de Archivos).

El sistema a construir simula un cluster[1] dedicado al entrenamiento de modelos de inteligencia artificial. Los usuarios envían trabajos de entrenamiento (de ahora en más, “Jobs”) que quedan encolados hasta que el cluster tenga recursos para atenderlos. Un Job es un programa que repite muchas veces un mismo ciclo de trabajo: trae un lote de datos, lo procesa hacia adelante a través de las capas del modelo, recorre esas mismas capas hacia atrás calculando las correcciones, y por último

[1] Conjunto de computadoras interconectadas que trabajan colaborativamente como si fuesen un único equipo

aplica esas correcciones sobre el modelo. Para hacerlo, cada Job reserva su espacio de trabajo en la memoria de la Placa y opera sobre él durante toda su ejecución.

Dos características de este entorno condicionan el diseño del sistema. La primera es que el espacio de trabajo de un Job suele ser más grande que la memoria disponible en la Placa, y varios Jobs conviven en ella al mismo tiempo. La segunda es que un entrenamiento puede durar días, por lo que un Job debe poder guardar su estado y retomarlo más adelante, incluso después de una caída del sistema.

Los componentes del sistema serán:

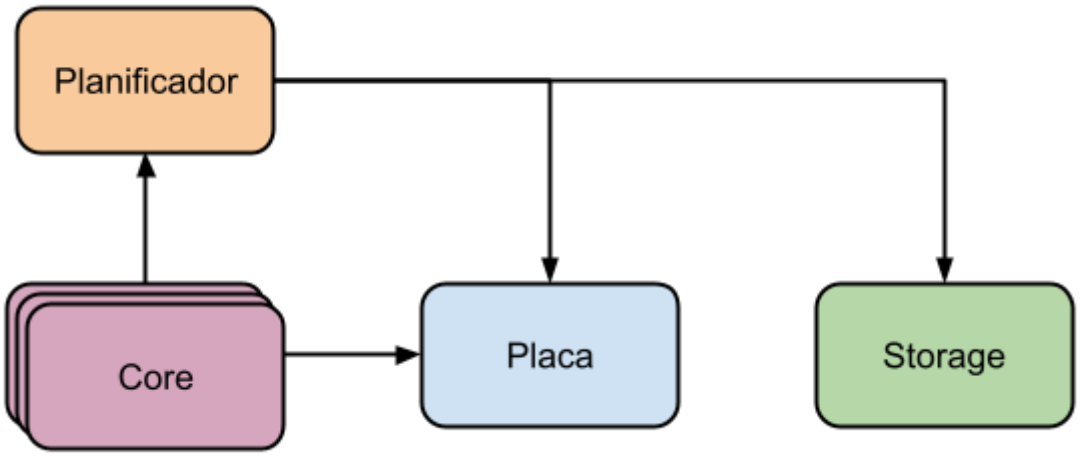
- Un proceso que administra la cola de Jobs y planifica su ejecución (Planificador).
- Uno o más procesos que ejecutan las instrucciones de los Jobs (Core).
- Un proceso que administra la memoria sobre la que los Jobs alojan su espacio de trabajo (la Placa).
- Un proceso que persiste los Checkpoints de los modelos (el Storage).

Para el desarrollo del mismo se decidió la creación de un sistema bajo la metodología Iterativa Incremental donde se solicitarán en una primera instancia la implementación de ciertos módulos para luego poder realizar una integración total con los restantes.

Recomendamos seguir el lineamiento de los distintos puntos de control que se detallan al final de este documento para su desarrollo. Estos puntos están planificados y estructurados para que sean desarrollados a medida y en paralelo a los contenidos que se ven en la parte teórica de la materia. *Cabe aclarar que esto es un lineamiento propuesto por la cátedra y no implica impedimento alguno para el alumno de realizar el desarrollo en otro orden diferente al especificado.*

Arquitectura del sistema

Este trabajo práctico correrá una serie de módulos los cuales deberán ser capaces de correr en diferentes computadoras y/o máquinas virtuales.



NOTAS:

- El sentido de las flechas indica dependencias entre módulos. Ejemplo: al momento de iniciar la ejecución de un Core es necesario contar con el Planificador y la Placa previamente iniciados.
- Cada uno de estos Módulos será un programa realizado en lenguaje C, compilado y ejecutado en la Máquina Virtual, por lo que será un proceso real del sistema operativo.
- A lo largo de este enunciado llamaremos Job (con J mayúscula) a los trabajos simulados que se planificarán y ejecutarán dentro de la implementación de este trabajo práctico.

Aclaración importante

Desarrollar únicamente temas de conectividad, serialización o sincronización es insuficiente para poder entender y aplicar los distintos conceptos de la materia. Dicho caso será un motivo de desaprobación directa.

Módulo: Planificador

Dentro de este TP, el Planificador será el módulo encargado de administrar la cola de Jobs del cluster y de gestionar su planificación a lo largo de las diferentes pruebas que se tengan que realizar.

Lineamiento e Implementación

El Planificador contará con un primer parámetro que será el archivo de configuración y un segundo parámetro que será el Path al Job Inicial, el cual será el JID[2] 0 del sistema y a partir del cual se crearán los demás Jobs.

```
➜ ~ ./bin/planificador [Archivo Config] [Path Job Inicial]
```

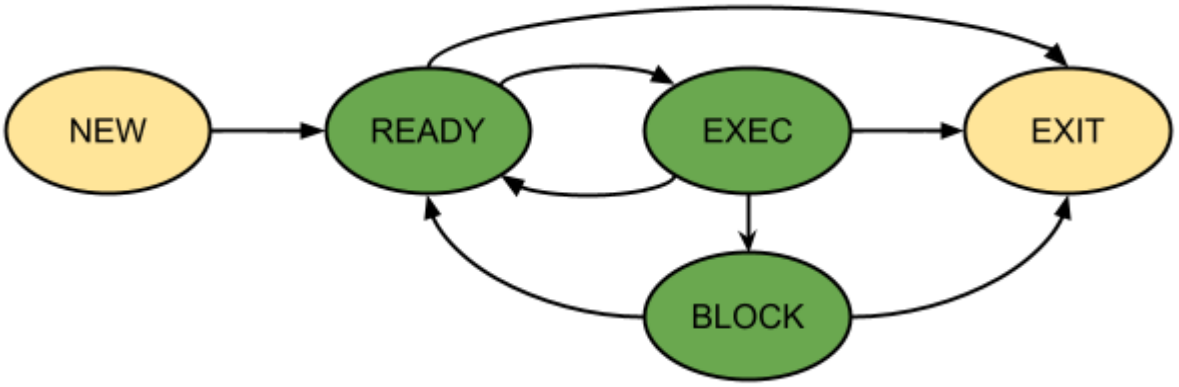
Al iniciar el módulo, el mismo se deberá conectar con la Placa y con el Storage, una vez que establezca dichas conexiones, deberá crear un servidor multihilo para atender de forma concurrente las peticiones de los Cores.

A lo largo de la ejecución se podrán desconectar y conectar nuevos Cores, por lo que se deberá mantener la escucha a nuevas conexiones siempre activa.

Planificación de Jobs

Una vez creado el servidor para escuchar las conexiones, el Planificador deberá cumplir con las tareas de planificación de largo y corto plazo detalladas más adelante. La implementación de las mismas es una decisión de diseño que deberá tomar el grupo, pero deberá ser capaz de gestionar un modelo de 5 estados:

[2] JID = “Job ID”. Un valor numérico utilizado para identificar unívocamente cada Job en el sistema.



Contexto de Ejecución

Por cada Job del sistema se deberá almacenar un contexto de ejecución que consiste, como mínimo, en una copia de todos los registros del Core inicializados en 0 al momento de crear el Job. El contexto se le enviará al Core al ponerlo a ejecutar, y este lo devolverá actualizado al liberarlo.

Planificación de Largo Plazo

Se encargará de ingresar Jobs del estado NEW al estado READY.

En caso de que el grado máximo de multiprogramación (definido por archivo de configuración) lo permita, los Jobs pasarán al estado READY mediante el algoritmo FIFO. Dado que los Jobs inician sin memoria asignada, este pasaje no requiere ninguna verificación sobre el espacio disponible en la Placa.

Cuando un Job finalice, se lo deberá pasar al estado EXIT, liberando las estructuras asociadas al mismo en todo el sistema.

Planificación de Corto Plazo

Vamos a estar utilizando uno de los tres posibles algoritmos de planificación definidos, los cuales van a ser FIFO, RR y HRRN. El algoritmo a utilizar será elegible por archivo de configuración y no cambiará a lo largo de una prueba. Para HRRN se tendrá un valor estimado inicial definido por archivo de configuración.

Page Fault

Cuando un Core informe un Page Fault, se deberá bloquear al Job y solicitarle a la Placa la carga de la página indicada. Una vez cargada, el Job deberá pasar al estado READY.

Atención de Syscalls

A lo largo de la ejecución de un Job dentro de un Core, el mismo puede solicitar al Planificador que ejecute diversas Syscalls. Dependiendo de cuál se trate, la Syscall podrá o no bloquear al Job que la solicitó.

Cuando un Core solicite una Syscall bloqueante, devolverá al Planificador el JID y el contexto de ejecución actualizado. El Planificador deberá guardar este contexto en las estructuras administrativas del Job y, al haberse liberado un Core, deberá invocar al planificador de corto plazo para asignarle un nuevo Job de la cola de READY.

Las Syscalls que no bloqueen al Job no liberarán el Core: una vez finalizadas se le deberá responder al Core que realizó la llamada para que continúe con su ejecución.

Las Syscalls que operan sobre la memoria del Job (LOAD_BATCH, LABEL, REPORT, SAVE_CHECKPOINT y LOAD_CHECKPOINT) llegarán con las direcciones físicas ya traducidas por el Core (un par [DireccionFisica;Tamaño] por cada página involucrada) y con sus páginas lockeadas en la Placa (page locking[3]). Una vez finalizada la operación, se deberá solicitar a la Placa el deslockeo de las páginas del Job antes de pasarlo al estado READY.

Ante la desconexión de un Core, el Job que estuviera ejecutando en él deberá volver al estado READY y se deberá solicitar a la Placa el deslockeo de sus páginas.

Jobs

INIT_JOB recibirá el nombre de un archivo de pseudocódigo y deberá crear un nuevo Job en el estado NEW, sin bloquear al Job que la solicitó.

EXIT deberá finalizar el Job que la solicitó, de acuerdo a lo definido en la Planificación de Largo Plazo.

Memoria

ALLOC y FREE se deberán derivar a la Placa y no bloquearán al Job. En el caso de ALLOC, se deberá responder al Core la dirección lógica informada por la Placa.

Servicios

LOAD_BATCH, LABEL y REPORT se deberán atender de acuerdo a lo definido en la sección de Servicios, bloqueando al Job hasta finalizar la operación.

Checkpoints

Las Syscalls de Checkpoints bloquearán al Job hasta finalizar la operación.

Para la syscall SAVE_CHECKPOINT, el Planificador recibirá del Core el nombre de un Checkpoint, un tamaño y las direcciones físicas correspondientes a ese rango. Se deberán leer de la memoria del Job dicha cantidad de bytes y enviarlos al Storage para que los persista con dicho nombre.

Para la syscall LOAD_CHECKPOINT, el Planificador recibirá del Core el nombre de un Checkpoint, un tamaño y las direcciones físicas correspondientes a ese rango. Se deberá solicitar su contenido al Storage y escribir dicha cantidad de bytes en la memoria del Job.

[3] El page locking consiste en marcar una página/marco temporalmente para que no pueda ser removida de la memoria por un reemplazo de página.

Para la syscall DELETE_CHECKPOINT, el Planificador recibirá del Core el nombre de un Checkpoint y deberá solicitar su eliminación al Storage.

Servicios

El Planificador deberá implementar 3 Servicios propios: Loader, Labeler y Logger. Cada uno atenderá una única petición por vez, bajo el algoritmo FIFO. En todos los casos se deberá bloquear al Job que solicitó la operación y mantener una cola de espera por cada Servicio.

Loader

Para el Loader, syscall LOAD_BATCH, el Planificador recibirá del Core el nombre de un archivo[4], un tamaño y las direcciones físicas correspondientes a ese rango. Se deberán leer dicha cantidad de bytes de ese archivo, ubicado en el directorio definido por archivo de configuración, y escribirlos en la memoria del Job. Esta operación tendrá un retardo definido por archivo de configuración.

Labeler

Para el Labeler, syscall LABEL, el Planificador recibirá del Core un tamaño y las direcciones físicas correspondientes a ese rango. Se le deberá pedir al usuario el ingreso de dicha cantidad de caracteres por teclado y, una vez recibidos, escribirlos en la memoria del Job.

Logger

Para el Logger, syscall REPORT, el Planificador recibirá del Core un tamaño y las direcciones físicas correspondientes a ese rango. Se deberán leer de la memoria del Job dicha cantidad de bytes y escribirlos en formato ASCII Hexadecimal en el archivo de reportes definido por archivo de configuración, que deberá ser distinto del archivo de log del módulo.

Estadísticas

Sobre cada Job ejecutado, se deberán calcular las siguientes estadísticas:

- Tiempo total de ejecución (desde el inicio a su finalización)
- Tiempo acumulado de espera (considerando solo el estado READY)
- Tiempo acumulado para ingresar (considerando solo el estado NEW)
- Tamaño máximo de memoria dinámica ocupada en un momento dado
- Cantidad acumulada de syscalls solicitadas
- Cantidad acumulada de page faults generados
- Cantidad acumulada de transiciones READY => EXEC

Las mismas deberán ser impresas en el Log cuando el mismo finalice.

[4] Este archivo se encontrará dentro del sistema de archivos sobre el cual ejecutará nuestro Planificador.

Logs mínimos y obligatorios

Conexión recibida: “## Módulo: <NOMBRE_MODULO_CONECTADO>”

Creación de Job: “## (<JID>) Se crea el Job - Estado: NEW”

Estimación: “## (<JID>) - Prioridad HRRN calculada: <PRIORIDAD> - Estimación próxima ráfaga: <ESTIMACION>”

Syscall recibida: “## (<JID>) - Solicitó syscall: <NOMBRE_SYSCALL>”

Cambio de Estado: “## (<JID>) Pasa del estado <ESTADO_ANTERIOR> al estado <ESTADO_ACTUAL>”

Inicio de Servicio: “## (<JID>) inicia <NOMBRE_SERVICIO>”

Fin de Servicio: “## (<JID>) finalizó <NOMBRE_SERVICIO> y pasa a READY”

Acceso a memoria: “## (<JID>) - <Lectura/Escritura> - Dirección Física: <DIRECCION_FISICA> - Tamaño: <TAMAÑO>”

Deslockeo de páginas: “## (<JID>) - Deslockeo de páginas”

Solicitud del Labeler: “## (<JID>) - Ingrese <CANTIDAD> caracteres:”

Reporte del Logger: “## (<JID>) - <CONTENIDO>”, en el archivo de reportes

Desalojo por fin de Quantum: “## (<JID>) - Desalojado por fin de quantum”

Fin de Job: “## (<JID>) finalizó su ejecución con motivo de <MOTIVO>. Estadísticas: <...>”

Archivo de Configuración

Campo	Tipo	Descripción
PUERTO_ESCUCHA	Número	Puerto utilizado para el servidor
IP_PLACA	String	IP del modulo Placa
PUERTO_PLACA	Número	Puerto utilizado para la Placa
IP_STORAGE	String	IP del módulo Storage
PUERTO_STORAGE	Número	Puerto utilizado para el Storage
LOG_LEVEL	String	Nivel de detalle máximo a mostrar. Compatible con log_level_from_string()
ALGORITMO_PLANIFICACION	String	Indica el algoritmo de planificación de corto plazo a utilizar, los valores posibles son FIFO, RR

Campo	Tipo	Descripción
		y HRRN
RR_QUANTUM	Número	Tiempo en milisegundos que se debe esperar antes de finalizar el Quantum de un Job
ESTIMACION_INICIAL	Número	Tiempo en milisegundos de estimación inicial para el cálculo de la primera ráfaga en HRRN
HRRN_ALFA	Número	Alfa para el cálculo de la estimación de las ráfagas en HRRN
GRADO_MULTIPROGRAMACION	Número	Cantidad máxima de Jobs que pueden estar residentes en la Placa de forma simultánea
PATH_DATOS	String	Directorio donde se encuentran los archivos que lee el Loader
RETARDO_LOADER	Número	Tiempo en milisegundos que demora el Loader en atender una petición
PATH_REPORTES	String	Archivo en el que el Logger escribe los reportes

Ejemplo de Archivo de Configuración

```
PUERTO_ESCUCHA=8080
IP_PLACA=127.0.0.1
PUERTO_PLACA=8081
IP_STORAGE=127.0.0.1
PUERTO_STORAGE=8082
LOG_LEVEL=INFO
ALGORITMO_PLANIFICACION=HRRN
RR_QUANTUM=1500
ESTIMACION_INICIAL=10000
HRRN_ALFA=0.5
GRADO_MULTIPROGRAMACION=4
PATH_DATOS=/home/utnso/datos/
RETARDO_LOADER=500
PATH_REPORTES=/home/utnso/reportes.log
```

Módulo: Core

El Core es el módulo encargado de interpretar y ejecutar las instrucciones de los Jobs. Para ello, ejecutará un ciclo de instrucción simplificado que cuenta con los pasos: Fetch, Decode, Execute y Check Interrupt.

Lineamiento e Implementación

Cada Core contará con un identificador propio, recibido por parámetro.

```
➜ ~ ./bin/core [Archivo Config] [Identificador]
```

Para cada Core se deberá tener un archivo de log y un archivo de configuración independientes, que deberán contar con el identificador pasado por parámetro para poder identificar a qué Core corresponde el archivo de log que se está leyendo. Los Cores deberán conectarse al Planificador y a la Placa.

Una vez establecidas todas las conexiones, el Core se quedará a la espera de recibir de parte del Planificador un JID junto con su contexto de ejecución, para comenzar el primer ciclo de instrucción.

A la hora de ejecutar instrucciones que requieran interactuar directamente con la memoria, tendrá que traducir las direcciones lógicas a direcciones físicas. Para ello usará la MMU.

Registros del Core

En la implementación de nuestro Core, se utilizará una serie de registros para poder modelar la operatoria de una CPU real simplificada; es decir, vamos a contar con registros similares a los vistos en Arquitectura de Computadores y algunos registros creados por nosotros mismos a fin de poder facilitar las pruebas.

En la siguiente tabla está el detalle de los registros que deberá tener nuestro Core, en la cual estará detallado el tamaño del mismo y qué tipo de dato debe contener:

Registro	Tamaño	Tipo de Dato	Descripción
PC[5]	4 bytes	uint32_t	Program Counter, indica la próxima instrucción a ejecutar
P0	4 bytes	uint32_t	Contiene una dirección lógica de memoria
P1	4 bytes	uint32_t	Contiene una dirección lógica de memoria
P2	4 bytes	uint32_t	Contiene una dirección lógica de memoria
P3	4 bytes	uint32_t	Contiene una dirección lógica de memoria

[5] En caso de que se realice una modificación de este registro en una operación, se omitirá el paso de sumar 1 al final del ciclo de instrucción

Registro	Tamaño	Tipo de Dato	Descripción
P4	4 bytes	uint32_t	Contiene una dirección lógica de memoria
P5	4 bytes	uint32_t	Contiene una dirección lógica de memoria
AX	4 bytes	uint32_t	Registro Numérico de propósito general
BX	4 bytes	uint32_t	Registro Numérico de propósito general
CX	4 bytes	uint32_t	Registro Numérico de propósito general
DX	4 bytes	uint32_t	Registro Numérico de propósito general
E1	4 bytes	uint32_t	Contiene un operando de las instrucciones de entrenamiento
E2	4 bytes	uint32_t	Contiene un operando de las instrucciones de entrenamiento
E3	4 bytes	uint32_t	Contiene un operando de las instrucciones de entrenamiento

Ciclo de Instrucción

Fetch

La primera etapa del ciclo consiste en buscar la próxima instrucción a ejecutar. En este trabajo práctico cada instrucción deberá ser pedida a la Placa utilizando el Program Counter que representa el número de instrucción a buscar relativo al Job en ejecución.

Decode

Esta etapa consiste en interpretar qué instrucción es la que se va a ejecutar y si la misma requiere de una traducción de dirección lógica a dirección física.

MMU

A la hora de traducir direcciones lógicas a físicas (ambas expresadas siempre en decimal), el Core debe tomar en cuenta que el esquema de la memoria de este sistema es de Paginación, por lo tanto, las direcciones lógicas se interpretarán de la siguiente manera:

`[N° Página | Desplazamiento]`

Las fórmulas para extraer ambos componentes de una Dirección Lógica serán las siguientes:

nro_pagina = floor(dir_logica / tamaño_pagina)

desplazamiento = dir_logica % tamaño_pagina

Es posible que una petición de lectura o escritura abarque más de una página. Será responsabilidad del grupo dividir estas peticiones de acuerdo a las diferentes páginas involucradas.

Todas las traducciones que requiera una instrucción deberán resolverse antes de modificar la memoria o los registros del Job.

En caso de que la página a la que se quiere acceder no se encuentre en la memoria de la Placa, se deberá devolver el Job al Planificador con motivo de Page Fault, sin actualizar el Program Counter.

Las páginas obtenidas quedarán lockeadas en la Placa, por lo que el Core deberá informar su liberación al finalizar la instrucción o al devolver el Job al Planificador. Se exceptúan las Syscalls que operan sobre la memoria del Job, cuyas páginas quedarán lockeadas hasta que el Planificador informe su liberación.

Listado de instrucciones a interpretar

```
NOOP
EXIT
SET AX 6
SET BX 256
SET P0 1024
SUM P0 BX
SUB AX 1
JNZ AX 5
FORWARD P0 P1 P2 BX
BACKWARD P0 P1 P2 BX
UPDATE P2 P3 P0
ALLOC P0 3072
FREE P0
LOAD_BATCH lote1 P1 CX
LABEL P1 CX
REPORT P1 CX
SAVE_CHECKPOINT modelo7 P0 BX
LOAD_CHECKPOINT modelo7 P0 BX
DELETE_CHECKPOINT modelo7
INIT_JOB finetune
```

Las instrucciones detalladas previamente son a modo de ejemplo, su ejecución no necesariamente sigue lógica alguna ni funcionamiento correcto. Al momento de realizar las pruebas, ninguna instrucción contendrá errores sintácticos ni semánticos.

Execute

En este paso se deberá ejecutar lo correspondiente a cada instrucción:

Instrucciones básicas

- NOOP: Representa la instrucción No Operation, es decir, va a consumir solamente el tiempo del ciclo de instrucción.
- SET (Registro, Valor): Asigna al registro el valor pasado como parámetro, que podrá ser un número o el contenido de otro registro.
- SUM (Registro, Valor): Suma al registro el valor pasado como parámetro.
- SUB (Registro, Valor): Resta al registro el valor pasado como parámetro.
- JNZ (Registro, Instrucción): Si el valor del registro es distinto de cero, actualiza el program counter al número de instrucción pasada por parámetro.

Instrucciones de entrenamiento

Las tres próximas instrucciones hacen referencia a operaciones que se utilizan para el entrenamiento, pero para este trabajo práctico su implementación se representa con operaciones aritméticas simples sobre enteros de 4 bytes sin signo (uint32_t). Los registros E1, E2 y E3 se corresponden con los tres primeros parámetros de la instrucción, y el Core cargará en ellos el contenido de las direcciones apuntadas antes de ejecutar dicha instrucción. Solo deberá cargar los registros que esa instrucción vaya a consumir (leer).

- FORWARD (Registro Pesos, Registro Entrada, Registro Salida, Registro Acumulador)

```
1    acumulador = acumulador + E1 * E2
2    E3 = acumulador
```

- BACKWARD (Registro Pesos, Registro Activaciones, Registro Gradientes, Registro Acumulador)

```
1    acumulador = acumulador - E1 * E2
2    E3 = acumulador
```

- UPDATE (Registro Gradientes, Registro Estado, Registro Pesos)

```
1    E2 = (E2 + E1) / 2
2    E3 = E3 - E2
```

Finalizada la ejecución, los registros de entrenamiento que hayan sido modificados (escritos) en esa instrucción deberán escribirse en la memoria del Job.

Instrucciones de tipo syscall

Las siguientes instrucciones se considerarán Syscalls, ya que las mismas no pueden ser resueltas por el Core y depende de la acción del Planificador para su realización. A diferencia de la vida real donde la llamada es a una única instrucción, para simplificar la comprensión de los scripts, vamos a utilizar un nombre diferente para cada Syscall. La descripción de lo que hace cada una va a estar detallada en el Planificador.

Las syscalls, al depender del Planificador, deberán ser solicitadas al mismo enviándole el JID y los parámetros correspondientes. Aquellas que bloqueen al Job deberán además liberar el Core y devolver el contexto de ejecución. Cuando la Syscall opere sobre la memoria del Job, el Core deberá traducir el rango indicado antes de devolver el Job y enviar al Planificador las direcciones físicas resultantes.

INIT_JOB, ALLOC y FREE **no bloquean al Job**: el Core deberá esperar la respuesta del Planificador y continuar con el siguiente ciclo de instrucción. En el caso de ALLOC, deberá guardar en el Registro Dirección Lógica la dirección recibida.

- INIT_JOB (Archivo de instrucciones)
- ALLOC (Registro Dirección Lógica, Tamaño)
- FREE (Registro Dirección Lógica)
- LOAD_BATCH (Nombre de archivo, Registro Dirección Lógica, Registro Tamaño)
- LABEL (Registro Dirección Lógica, Registro Tamaño)
- REPORT (Registro Dirección Lógica, Registro Tamaño)
- SAVE_CHECKPOINT (Nombre, Registro Dirección Lógica, Registro Tamaño)
- LOAD_CHECKPOINT (Nombre, Registro Dirección Lógica, Registro Tamaño)
- DELETE_CHECKPOINT (Nombre)
- EXIT

Es importante tener en cuenta que, al finalizar el ciclo de instrucción, el Program Counter (PC) deberá ser actualizado sumándole 1, siempre y cuando este no haya sido modificado por la instrucción ejecutada anteriormente.

Check Interrupt

En este momento, se deberá chequear si el Planificador nos envió una interrupción al Job que se está ejecutando. En caso afirmativo, se le devuelve el JID y el contexto de ejecución al Planificador con el motivo de la interrupción. Caso contrario, continúa con el siguiente ciclo de instrucción.

Ejemplo de un Job

En este archivo encontrarán un ejemplo de un Job que realiza un paso de entrenamiento.

Logs mínimos y obligatorios

Conexión establecida: “## Conectado a <Planificador/Placa> exitosamente”

Fetch Instrucción: “## JID: <JID> - FETCH - Program Counter: <PROGRAM_COUNTER>”.

Interrupción Recibida: “## Interrupción recibida”.

Instrucción Ejecutada: “## JID: <JID> - Ejecutando: <INSTRUCCION> - <PARAMETROS>”.

Page Fault: “## JID: <JID> - Page Fault - Página: <NRO_PAGINA>”.

Lectura/Escritura Memoria: “## JID: <JID> - Acción: <Lectura/Escritura> - Dirección Física: <DIRECCION_FISICA> - Tamaño: <TAMAÑO>”.

Archivo de Configuración

Campo	Tipo	Descripción
LOG_LEVEL	String	Nivel de detalle máximo a mostrar. Compatible con log_level_from_string()
IP_PLANIFICADOR	String	IP del módulo Planificador
PUERTO_PLANIFICADOR	Número	Puerto utilizado para el Planificador
IP_PLACA	String	IP del módulo Placa
PUERTO_PLACA	Número	Puerto utilizado para la Placa

Ejemplo de Archivo de Configuración

```
LOG_LEVEL=INFO
IP_PLANIFICADOR=127.0.0.1
PUERTO_PLANIFICADOR=8080
IP_PLACA=127.0.0.1
PUERTO_PLACA=8081
```

Módulo: Placa

La Placa es el módulo encargado de administrar la memoria del sistema. En ella se almacenan las instrucciones de los Jobs junto con el espacio de trabajo sobre el que estos operan.

Lineamiento e Implementación

Al iniciar creará un servidor multihilo que se encargará de gestionar de manera concurrente las peticiones del Planificador y de los Cores. A lo largo de la ejecución se podrán conectar nuevos Cores, por lo que se deberá mantener la escucha a nuevas conexiones siempre activa.

```
➜ ~ ./bin/placa [Archivo Config]
```

Memoria de Instrucciones

En esta sección la Placa contendrá las instrucciones de los Jobs, que le serán solicitadas por los diferentes Cores. Dicho espacio será ilimitado en capacidad (a diferencia de la memoria de usuario)

Los archivos de pseudocódigo serán archivos de texto que contendrán dichas instrucciones, separadas por el caracter '\n'. Por cada Job del sistema se tendrá uno, para lo que cada grupo deberá implementar una estructura que le permita asociar qué Job tiene asociado qué archivo.

La Placa deberá ser capaz de enviar la instrucción correspondiente a cada pedido del Core sin haber una restricción específica de cómo se deben obtener las mismas.

Memoria de Usuario

Esquema de memoria y Estructuras

La memoria de datos de la Placa utilizará un esquema de paginación con memoria virtual, con asignación dinámica y alcance global. El tamaño total y tamaño de página estarán definidos por archivo de configuración.

Se deberá implementar paginación bajo demanda: los Jobs empezarán sin marcos asignados, y una página pasará a ocupar uno recién cuando el Job la acceda.

La Placa contará como mínimo con dos estructuras:

- Un espacio contiguo de memoria (representado por un void*), que será la memoria de datos del sistema.
- Una tabla de páginas por cada Job.

Las tablas de páginas son estructuras administrativas que no deben guardarse dentro de la memoria de datos.

El Core deberá solicitarle a la Placa el marco correspondiente a cada página a la que necesite acceder.

El Offload

El Offload es un archivo administrado por la Placa, cuyo path y tamaño se definen por archivo de configuración. En él se almacenan las páginas de los Jobs que no se encuentran en la memoria de la Placa.

Al imitar los accesos a disco, el Offload deberá implementarse como si fuera un único hilo de ejecución: no se podrán realizar dos o más lecturas o escrituras en paralelo.

Reemplazo de páginas

Cuando se deba cargar una página en la memoria de la Placa y no existan marcos libres, se deberá seleccionar una página víctima entre todas las páginas del conjunto residente[6] que no se encuentren lockeadas, sin importar a qué Job pertenezcan. El algoritmo de reemplazo se definirá por archivo de configuración, pudiendo ser LRU o Clock Modificado.

Operaciones

La Placa, al ser el módulo que gestiona la memoria del sistema y de los Jobs, puede aceptar una serie de operaciones que describiremos a continuación.

Creación de Job

Para esta operación se va a recibir un JID y el nombre de un archivo de pseudocódigo, relativo al path base configurado por archivo de configuración. Con los datos recibidos, la Placa deberá crear las estructuras administrativas necesarias para el Job.

Asignación de espacio

Al momento de asignar espacio la Placa recibirá como parámetro el JID del Job y el tamaño en bytes a asignar, y deberá responder la dirección lógica en la que comienza el espacio asignado. Cada asignación deberá comenzar al inicio de una página.

Se deberá reservar en el Offload el lugar necesario para respaldar las páginas asignadas. En caso de no contar con espacio suficiente, la operación no se podrá realizar y se le deberá informar al Planificador para que finalice el Job.

Liberación de espacio

Al momento de liberar espacio se recibirá el JID del Job y la dirección lógica en la que comienza un espacio previamente asignado. Se deberán liberar los marcos ocupados por sus páginas y el lugar que estas tenían reservado en el Offload.

[6] Conjunto residente: set de páginas de un Job, actualmente cargadas en la Placa

Obtención de marco

Esta operación es solicitada por el Core al momento de realizar la traducción de la MMU. Recibirá como parámetro el JID y el número de página. La Placa deberá consultar la Tabla de Páginas del Job:

- Si la página se encuentra residente en memoria, le responderá al Core con el número de marco correspondiente. La página pasará a estar lockeada.
- Si la página no se encuentra residente, le responderá al Core indicando que ocurrió un Page Fault.

Carga de página

Esta operación será solicitada por el Planificador cuando un Core informe un Page Fault. Se recibirá el JID del Job y un número de página, y se deberá cargar la misma en la memoria de la Placa.

Lectura de datos

Esta operación puede llegar de un Core o del Planificador, y en ambos casos se recibirá el JID, una dirección física y un tamaño a leer. Se deberá responder el contenido del rango pedido.

Escritura de datos

Esta operación recibirá los mismos parámetros que la de lectura, junto con el contenido a escribir.

Deslockeo de páginas

Esta operación será enviada por el Core cada vez que finalice con éxito un ciclo de instrucción, o cuando devuelva el contexto al Planificador (por ejemplo, al sufrir un Page Fault). En el caso de las Syscalls que operan sobre la memoria del Job, será enviada por el Planificador al finalizar la operación.

La Placa recibirá el JID y deberá liberar el estado de lockeo de todas sus páginas, permitiendo que los marcos asignados vuelvan a ser elegibles como víctimas de reemplazo.

Finalización de Job

Esta operación se utiliza para finalizar un Job, por lo que solamente va a recibir un JID. A partir del mismo se deberán liberar todos los marcos que tuviera ocupados, el lugar que tuviera reservado en el Offload y todas las estructuras asociadas al mismo.

Comandos locales

La Placa deberá implementar una consola por stdin, soportando los siguientes comandos. Estos responderán mostrando información en un formato “human-readable”:

- INFO: mostrar porcentaje de memoria/offload ocupada, cantidad de frames libres/lockeados/totales, cantidad de Jobs actuales

- TLS: listar todos los Jobs registrados, indicando para cada uno: ID, cantidad de páginas del conjunto residente y cantidad de páginas totales

Se podrán agregar otros comandos que se consideren necesarios, siempre y cuando no modifiquen ni contradigan el funcionamiento detallado en el módulo.

Logs mínimos y obligatorios

Conexión recibida: "## Módulo: <NOMBRE_MODULO_CONECTADO>"

Creación de Job: "## JID: <JID> - Job Creado"

Obtener instrucción: "## JID: <JID> - Obtener instrucción: <PC> - Instrucción: <INSTRUCCIÓN> <...ARGS>"

Asignación / Liberación de espacio: "## JID: <JID> - <Asignación/Liberación> - Dirección Lógica: <DIRECCIÓN_LÓGICA> - Tamaño: <TAMAÑO>"

Subida de página a memoria: "## JID: <JID> - Subida a Memoria - Página: <NRO_PÁGINA> - Marco: <NRO_MARCO>"

Bajada de página al Offload: "## JID: <JID> - Bajada al Offload - Página: <NRO_PÁGINA>"

Reemplazo de página: "## Reemplazo - Marco: <NRO_MARCO> - JID Víctima: <JID> - Página Víctima: <NRO_PÁGINA>"

Lectura/Escritura en espacio de usuario: "## JID: <JID> - <Lectura/Escritura> - Dir. Física: <DIRECCION_FISICA> - Tamaño: <TAMAÑO>"

Archivo de Configuración

Campo	Tipo	Descripción
PUERTO_ESCUCHA	Número	Puerto utilizado para el servidor
LOG_LEVEL	String	Nivel de detalle máximo a mostrar. Compatible con log_level_from_string()
TAM_MEMORIA	Número	Tamaño expresado en bytes de la memoria de datos (valores potencia de 2)
TAM_PAGINA	Número	Tamaño de las páginas en bytes (valores potencia de 2)
RETARDO_MEMORIA	Número	Tiempo en milisegundos que se deberá esperar para dar una respuesta
ALGORITMO_REEMPLAZO	String	Algoritmo de reemplazo de páginas, los valores

Campo	Tipo	Descripción
		posibles son LRU y CLOCK-M
PATH_INSTRUCCIONES	String	Carpeta donde se encuentran los archivos de pseudocódigo de los Jobs
PATH_OFFLOAD	String	Path del archivo de Offload
TAM_OFFLOAD	Número	Tamaño expresado en bytes del archivo de Offload
RETARDO_OFFLOAD	Número	Tiempo en milisegundos que se deberá esperar para cada operación del Offload (leer/escribir)

Ejemplo de Archivo de Configuración

```
PUERTO_ESCUCHA=8080
LOG_LEVEL=INFO
TAM_MEMORIA=4096
TAM_PAGINA=64
RETARDO_MEMORIA=10
ALGORITMO_REEMPLAZO=CLOCK-M
PATH_INSTRUCCIONES=/home/utnso/scripts/
PATH_OFFLOAD=/home/utnso/offload.dat
TAM_OFFLOAD=65536
RETARDO_OFFLOAD=100
```

Módulo: Storage

El Storage es el módulo encargado de persistir los Checkpoints de los Jobs. Los mismos sobreviven a la finalización del Job que los generó, por lo que un Job puede cargar un Checkpoint creado por otro.

Lineamiento e Implementación

Al iniciar creará un servidor multihilo que deberá ser capaz de atender las peticiones del Planificador de manera concurrente.

```
➜ ~ ./bin/storage [Archivo Config]
```

Toda la información que administra el Storage deberá persistirse dentro del directorio definido por archivo de configuración, de manera que su contenido sobreviva entre ejecuciones del módulo.

Implementación

El Storage implementará un sistema de archivos cuya especificación será una variante de “FAT32”, la cual daremos en llamar “FAT32_TRAIN”. Junto a él se encontrará el journal, que se detalla en la sección Journaling.

Estructura interna del sistema de archivos

El volumen de dicho sistema se persistirá en un archivo real del sistema operativo anfitrión (Linux en nuestro caso). El mismo se dividirá en bloques de igual tamaño, numerados a partir del 0. La cantidad de bloques y el tamaño de cada uno se definen por archivo de configuración.

Sus bloques se reparten entre cuatro regiones:

- El superbloque, que ocupa el bloque 0.
- La tabla FAT, cuyo tamaño depende de la cantidad de bloques y del tamaño del puntero.
- Un único directorio raíz, que ocupa la cantidad de bloques definida por archivo de configuración.
- Los bloques de datos, que son todos los restantes.

Si al iniciar el módulo el archivo del filesystem no existe, deberá ser creado y “formateado”: deberá escribir el superbloque, la tabla FAT con la totalidad de sus bloques de datos libres y el directorio con todas sus entradas libres.

Superbloque

Ocupa el bloque 0 y describe el formato del filesystem. Deberá contar exactamente con la siguiente estructura de datos contigua:

- Firma (Magic Number): 11 bytes (Cadena de caracteres estática, "FAT32_TRAIN", para identificar que es un filesystem válido).
- Tamaño de bloque: 4 bytes (uint32_t).

- Cantidad de bloques del filesystem: 4 bytes (uint32_t).
- Cantidad de bloques del directorio: 4 bytes (uint32_t).

El resto del bloque 0 (hasta completar el TAM_BLOQUE definido) deberá llenarse con padding (ceros) y queda reservado para uso futuro.

File Allocation Table

Contiene una entrada por cada bloque del filesystem, representada por un dato de tipo uint32_t, por lo que su tamaño será la cantidad de bloques por sizeof(uint32_t).

Cada entrada indica cuál es el bloque siguiente del Checkpoint al que pertenece. De sus 32 bits sólo los 28 menos significativos forman el número de bloque: los 4 más significativos se encuentran reservados.

El valor 0 representa que el bloque se encuentra libre, y el valor 0x0FFFFFFF representa el fin del Checkpoint.

Las entradas correspondientes a bloques reservados deberán marcarse como 0x0FFFFFFE, de manera que nunca se los considere libres.

La tabla ocupará tantos bloques como sea necesario para almacenar sus entradas, completando con ceros el resto del último bloque en caso de no completarlo.

Directorio

Contiene una entrada por cada Checkpoint (archivo) que exista en el filesystem, ocupando la región de manera contigua desde su primer byte. Cada entrada deberá contar exactamente con la siguiente estructura de datos:

- Estado de la entrada: 1 byte (uint8_t, 0 si se encuentra libre y 1 si se encuentra ocupada).
- Nombre del Checkpoint: 23 bytes (cadena de caracteres terminada en '\0').
- Tamaño del Checkpoint expresado en bytes: 4 bytes (uint32_t).
- Primer bloque de datos: 4 bytes (uint32_t).

En caso de que el directorio se encuentre lleno, deberá informarse con un error.

Operaciones

El Storage deberá atender las siguientes operaciones, que serán solicitadas por el Planificador.

Guardar un Checkpoint

Se recibirá el nombre de un Checkpoint junto con el contenido a persistir. Si el Checkpoint no existe se deberá crear su entrada en el directorio, y si ya existe su contenido anterior será reemplazado.

Se le deberán asignar tantos bloques como sean necesarios para direccionar el tamaño recibido y liberar aquellos que dejen de ser necesarios.

En caso de no contar con espacio suficiente, se deberá informar dicha situación al Planificador y se finalizará el Job.

Cargar un Checkpoint

Se recibirá el nombre de un Checkpoint y se deberá responder su contenido completo. En caso de que el Checkpoint no exista, se deberá informar dicha situación al Planificador y se finalizará el Job.

Eliminar un Checkpoint

Se recibirá el nombre de un Checkpoint y se deberá liberar el espacio que ocupaba: su entrada del directorio deberá marcarse como libre, y todos los bloques de su cadena deberán marcarse como libres en la tabla FAT.

En caso de que el Checkpoint no exista, se deberá informar dicha situación al Planificador, pero no se debe finalizar el Job.

Journaling

Con el objetivo de preservar la coherencia de los datos ante posibles fallas, el sistema de archivos utilizará un sistema de registro (Journaling) para intentar garantizar la integridad de la información. Para ello, las operaciones Guardar y Eliminar un Checkpoint deberán quedar asentadas en dicho registro antes de aplicarse efectivamente, de manera que el módulo pueda recuperar un estado consistente luego de una finalización abrupta.

El journal

El journal es un archivo real (distinto al archivo del volumen del filesystem) de escritura secuencial, en el que se describen anticipadamente todas las operaciones a realizarse luego en el filesystem.

Por cada operación se registrarán todas las modificaciones que la integran, agrupadas en lo que denominaremos un “asiento”.

Un asiento deberá contener un identificador único, las modificaciones concretas que la operación produce (bloques de datos, entradas de la FAT y del directorio) con el contenido que queda en cada una, y unas marcas que indican el grado de completitud de la operación que comprende.

Procedimiento de uso

Primer paso: escritura en el journal

Se escribirán en el journal todas las modificaciones que comprendan dicha operación. Una vez completado el asiento, se agregará una marca de tipo “Commit” indicando el mismo como completado.

El journal deberá escribirse utilizando fwrite seguido de fflush y fsync, de manera de garantizar que el asiento se encuentre efectivamente en disco antes de continuar.

Segundo paso: escritura en filesystem

Una vez completado el paso anterior, se aplicarán efectivamente todas las modificaciones del asiento recientemente completado en el sistema de archivos. Una vez aplicada toda la operación, se agregará una nueva marca de tipo "Applied" indicando que el asiento está aplicado.

El uso de fflush y fsync en este paso es opcional, pero en caso de usarlos deberán poder desactivarse por configuración.

Recuperación ante fallas

Al iniciar, el Storage deberá recorrer cronológicamente el journal en la búsqueda de operaciones que hayan quedado inconclusas, accionando de la siguiente manera por cada asiento:

- Un asiento sin marca "commit" será descartado
- Un asiento con ambas marcas presentes ("commit" y "applied") será descartado
- Un asiento con marca "commit", pero sin marca "applied" será re-aplicado en el filesystem en su totalidad

Las operaciones deberán ser idempotentes (volver a aplicar un asiento que ya había sido aplicado no puede alterar el resultado).

Este procedimiento será evaluado finalizando el módulo de manera abrupta en medio de una operación y volviéndolo a iniciar.

Ejemplo de un Asiento en el Journal

Supongamos que se solicita la operación *SAVE_CHECKPOINT modelo7*, y el Storage decide asignarle los bloques de datos 0x0000000F y 0x00000010, utilizando la tercera entrada de directorio. La estructura de ese asiento en el Journal *podría* verse de la siguiente manera:

```
RECORD_STARTED: 1
OP_DIR: Directorio 3 - Checkpoint "modelo7" - Tamaño 60 - Bloque Inicial 0x0000000F
OP_FAT: Entrada 0x0000000F -> Valor 0x00000010
OP_FAT: Entrada 0x00000010 -> Valor 0x0FFFFFFF[7]
OP_WRITE: Bloque 0x0000000F - Contenido: <...>[8]
OP_WRITE: Bloque 0x00000010 - Contenido: <...>
RECORD_COMMITED 1
RECORD_APPLIED 1
```

[7] Un puntero a bloque deberá representarse en hexadecimal
[8] Dado que los bloques contienen "bytes", se sugiere persistirlo en caracteres hexadecimales

Considerando el ejemplo provisto, se aclara que la estructura definitiva del journal quedará a criterio de cada grupo, pero deberá respetarse el formato de un archivo de texto.

Comandos locales

El Storage deberá implementar una consola por stdin, soportando los siguientes comandos. Estos responderán mostrando información en un formato “human-readable”:

- INFO: mostrar el contenido actual del superbloque
- LS: listar todas las entradas de directorio, con todos sus atributos
- CAT <CheckpointName> <cantBytes>: mostrar el contenido de los primeros “cantBytes” en hexadecimal
- WRITE <CheckpointName> <hexaContent>: escribe o crea un archivo con el contenido especificado en hasta 20 bytes hexadecimales
- RM <CheckpointName>: borra el archivo especificado

Se podrán agregar otros comandos que se consideren necesarios, siempre y cuando no modifiquen ni contradigan el funcionamiento detallado en el módulo.

Logs mínimos y obligatorios

Conexión recibida: “## Módulo: <NOMBRE_MODULO_CONECTADO>”

Solicitud recibida: “## Solicitud [Guardar|Cargar|Eliminacion] Checkpoint: <NOMBRE>”

Solicitud completada: “## [Guardado|Carga|Eliminacion] Checkpoint: <NOMBRE> - Tamaño: <TAMAÑO> completada”

Errores: "## ERROR - <Falta de espacio en FAT / Directorio lleno> al intentar guardar: <NOMBRE>"

Acceso a File Allocation Table: “## Acceso FAT - Entrada: <NRO_ENTRADA> - Valor: 0x<VALOR>”

Acceso a Bloque de datos: “## Acceso Bloque - Checkpoint: <NOMBRE> - Bloque: <NRO_BLOQUE>”

Journal: “## Asiento iniciado: <ID_ASIENTO>”

Journal: “## Asiento: <ID_ASIENTO> - Modificacion: <MODIFICACION>”

Journal: “## Nueva marca en asiento: <ID_ASIENTO>: - [COMMIT|APPLIED]”

Recuperación: “## Recuperación - Asiento: <ID_ASIENTO> - <APLICADO / DESCARTADO>”

Archivo de Configuración

Campo	Tipo	Descripción
PUERTO_ESCUCHA	Número	Puerto utilizado para el servidor

Campo	Tipo	Descripción
LOG_LEVEL	String	Nivel de detalle máximo a mostrar. Compatible con log_level_from_string()
PATH_STORAGE	String	Carpeta donde se encuentran los archivos del filesystem y del journal
CANT_BLOQUES	Número	Cantidad de bloques del filesystem
TAM_BLOQUE	Número	Tamaño de los bloques en bytes (valores potencia de 2, comenzando en 32)
BLOQUES_DIRECTORIO	Número	Cantidad de bloques que ocupa el directorio
RETARDO_ACCESO_BLOQUE	Número	Tiempo en milisegundos que se deberá esperar ante cada acceso a cualquier bloque del volumen (conteniendo datos, directorio o entradas de la FAT)
RETARDO_COMMIT	Número	Tiempo en milisegundos que se deberá esperar *antes* de marcar cada asiento como "COMMITED"

Ejemplo de Archivo de Configuración

```
PUERTO_ESCUCHA=8080
LOG_LEVEL=INFO
PATH_STORAGE=/home/utnso/storage/
CANT_BLOQUES=1024
TAM_BLOQUE=64
BLOQUES_DIRECTORIO=64
RETARDO_ACCESO_BLOQUE=100
RETARDO_COMMIT=100
```

Descripción de las entregas

Debido al orden en que se enseñan los temas de la materia en clase, los checks de control están diseñados para que se pueda realizar el trabajo práctico de manera iterativa incremental tomando en cuenta los conceptos aprendidos hasta el momento de cada check de control.

Check de Control Obligatorio 1: Conexiones y Serialización

Fecha: 12/09

Objetivos:

- Garantizar que los alumnos hayan implementado la arquitectura de red y ruteo de mensajes básica.
- El Planificador, la Placa y el Storage deben poder inicializar sus servidores multihilo.
- El Core debe lograr conectarse de forma exitosa al Planificador y a la Placa, realizar un Handshake inicial para identificarse y enviar/recibir un paquete de mensajes serializado como prueba.

Cómo será testeado:

Se levantarán los 4 procesos de forma aislada y se validará a través de los logs obligatorios que los Handshakes iniciales de identificación se realizan de forma correcta y que los paquetes se envían/reciben sin cuelgues ni caídas de conexión.

Check de Control Obligatorio 2: Planificación y Ejecución Básica

Fecha: 10/10

Objetivos:

- Integrar el ciclo de instrucción en el Core (aún sin memoria de datos ni MMU).
- El Planificador deberá gestionar la cola de Jobs y demostrar el correcto cambio de contexto (**Context Switch**) evaluando los algoritmos de planificación a corto plazo (*FIFO, RR*).

Cómo será testeado:

Se solicitará la ejecución concurrente de varios scripts de pseudocódigo matemáticos o de tipo *loops contadores* (*NOOP, SET, SUM, SUB, JNZ*).
Se verificará en los logs del Planificador que los Jobs respeten los quantums y prioridades, alternando de forma correcta entre los estados *READY, EXEC y EXIT*.

Check de Control Obligatorio 3: Memoria, Paginación y Servicios, Storage

Fecha: 31/10

Objetivos:

- Habilitar la MMU del Core para traducir direcciones y reportar **Page Faults**.
- En la Placa, implementar tablas de páginas, paginación bajo demanda, algoritmo de reemplazo y Offload (Swap).
- En el Planificador, implementar los servicios I/O bloqueantes.
- En el Storage, implementar el formateo del volumen y los comandos locales

Cómo será testeado:

Se ejecutará un script tipo *Mini Entrenamiento* que reserve memoria (*ALLOC*), lea datos simulados (*LOAD_BATCH*) y recorra ese espacio con las instrucciones de entrenamiento (*FORWARD, BACKWARD, UPDATE*).
Esto forzará el acceso a múltiples páginas generando **Page Faults**. Se validará en los logs el correcto envío de páginas al Offload y el pasaje de los Jobs por la cola de **BLOCKED**.

Check de Control Obligatorio 4: Storage, Filesystem y Persistencia

Fecha: 14/11

Objetivos:

- Storage: implementar todas las solicitudes que puedan llegar del Planificador
- Storage: implementar la funcionalidad de Journaling.

Cómo será testeado:

Se correrá un script que trabaje datos en memoria y los persista en disco usando *SAVE_CHECKPOINT*, seguido de un script independiente que recupere dicha sesión vía *LOAD_CHECKPOINT*.
Adicionalmente, se forzará la detención abrupta del proceso Storage (kill -9) en el medio de una operación de guardado; al reiniciarlo, se validará en los logs que el Journal logra restaurar el Filesystem a un estado íntegro y consistente.

Entregas Finales

Fechas: 28/11, 12/12, 19/12

Objetivos:

- Finalizar el desarrollo de todos los módulos.
- Probar de manera intensiva el TP en un entorno distribuido.
- Todos los componentes del TP ejecutan los requerimientos de forma integral.